



THE LAB CODER

Advanced Notes Part 1 (update 10 Sep 2026)

Notes: <http://tinyurl.com/TheLabAdvanced1>

Python

Python is a widely-used, interpreted, object-oriented, and high-level programming language with dynamic semantics, used for general-purpose programming.

It was created by Guido van Rossum and was first released on 20 February, 1991.

Some popular software development that had use Python extensively:

Spotify, Google, Instagram, YouTube, Pinterest,
Battlefield 2, Eve Online, World of Tanks, The Sims 4

Content

Page	
4	[3031] <u>Print</u>
9	[3032] <u>Variable</u>
18	[3032] <u>Operators</u>
25	[3033] <u>Type Casting</u>
28	[3033] <u>Input</u>
37	[3034] <u>If Statement</u>
50	[3035] <u>For-Loop</u>
57	[3036] <u>Good Programming Habits</u>
62	[3037] <u>While-Loop</u>
67	[3038] <u>List Basics</u>
79	[3038] <u>List Manipulation</u>
88	[3039] <u>String Manipulation</u>
95	[3039] <u>Miscellaneous</u>
110	[3039] <u>Practices and IDEs</u>

Click on titles

PRINT

PRINT

Print command is to display value or text on the screen.
Below is the *syntax* of a print command.

put the data to display here
↓
print()

Use single quote (') or double quote(") around the text you want to print.

Use single or double quotes (i.e. 'Hello world' or "Hello world")
↓ ↓
print("Hello world.")

PRINT

What if you need to display quotes in your text (e.g. Score "A" is good) ?

this will cause an error because the second quote is taken as a closing quote for the first and the third quote taken as the opening quote for the last

```
print( "Score "A" is good" )
```

Add a backslash (\) to mark a quote as part of the text as shown:

add a backslash before the text quotes

```
print( "Score \"A\" is good" )
```

Screen output

```
Score "A" is good
```

COMMENTING

As you write your program, you may want to insert a remark.
This can be done by using the hash sign **#**.

Examples:

```
# display hello world ← this line is your comment and it will not be run as a code  
print( "Hello world!" )
```

you can also put your comment beside a code but it is
recommended to use the above format instead

```
print( "Hello world!" ) # display hello world
```

SyntaxError:

You when receive the a value error message show below, it means the structure of your code is invalid.

```
SyntaxError: invalid syntax
```

For example, if you missed closing a bracket or a quote of a print statement, it will generate a syntax error.

```
print("Hello, world)
```

missing closing quote "

VARIABLE

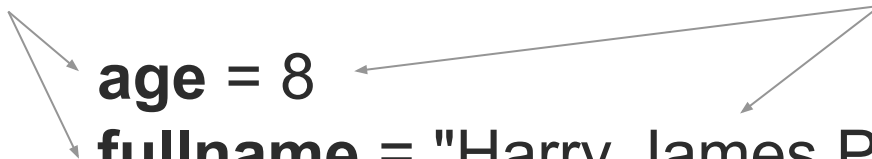
VARIABLE

A variable is used to store a value which you need to reference subsequently. To create a variable, give it a name and assign a value to it.

Examples:

your variables

values assigned to the variables



```
age = 8  
fullname = "Harry James Potter"
```

Note that it is always the RIGHT value saved to the LEFT.

VARIABLE NAME

The name of a variable must begin with either a letter or an underscore "_".
Followed by any letters, digits or underscores. No spaces are allowed.

Examples of valid names

name
num4
first_name
_length
_1stPlayer
_02
i

Examples of invalid names

first name <- space not allowed
1st_Player <- first character must be letter or underscore
circle.radius <- only letters, digits and underscore are allowed

Use ***meaningful*** names for variables. By convention, programmers use lower case for variables (although upper case is allowed) and words are separated with underscore.

VARIABLE DATA TYPE

Variable can store different data type.
Below are examples of 4 basic data types.

Data Type	Examples	
string	'Hello World!' "ABC 888"	← string(text). Must have open and close quotes. You can use either single quotes ' ' or double quotes " "
integer	123 , -123	← whole numbers (no decimal points)
float	1.23 , -5.0	← numbers with decimal points
boolean	True False	← only have True or False values

PRINTING VARIABLE (using comma separator)

Example:

```
number = 8
```

```
print( number )
```

```
print( "The number is" , number )
```

↑
Within quotes "
this is recognized as a text

↑
Without quotes,
this is recognized as a variable

↑
Place a comma between items.
A space will appear in the output.

Screen output

```
8  
The number is 8
```

PRINTING VARIABLE

You can use comma (,) or plus sign (+) to join different items in print command. A comma (,) will add a space between the items.

```
name = "Alan"  # variable stores string
number = 2     # variable stores integer
```

Join items using commas. Each comma will add a space to the output.

```
print( name , "has" , number , "dogs." )
```

Join **strings** using plus sign. Plus sign does not add a space.

```
print( name + "has" , number , "dogs." )
```

Space within quotes will appear in output.

```
print( name + " has" , number , "dogs." )
```

Spaces outside the quotes have no effect on the output.

```
print( name + "has" , number , "dogs." )
```

Screen output

Alan has 2 dogs.

Alanhas 2 dogs.

Alan has 2 dogs.

Alanhas 2 dogs.

PRINTING VARIABLE (using f-string)

Example:

```
number = 8
```

```
print( number )
```

```
print( f"The number is {number}" )
```

↑
Precede with an **f** followed
by single or double quote
(no space in between)

↑
Place variable inside
curly brackets

Screen output

```
8  
The number is 8
```

PRINTING VARIABLE

Printing variables with f-string formatting is another option that is easier.

```
name = "Alan"  # variable stores string  
number = 2     # variable stores integer
```

```
print( f"{name} has {number} dogs." )
```

Screen output

```
Alan has 2 dogs.
```

VARIABLE DATA TYPE

In Python, the same variable can store values of different data types while the program is running.

Example:

```
number = 8  
print( "number stores integer" , number )  
number = "eight"  
print( "Now number stores string", number )
```

Screen output

```
number stores integer 8  
Now number stores string eight
```

OPERATORS

ARITHMETIC OPERATORS

You can apply arithmetic operations on numbers and numeric variables (integer and float)

Operator	Description	Example x = 5 and y = 2	
+	Addition	x + y	(ans: 7)
-	Subtraction	x - y	(ans: 3)
-	Negative	-x	(ans: -5)
*	Multiplication	x * y	(ans: 10)
/	Division	x / y	(ans: 2.5)
%	Modulus (remainder)	x % y	(ans: 1)
//	Floor division (quotient)	x // y	(ans: 2)
**	Exponentiation (the power of)	x ** y	(ans: 25)
=	Equal (assignment)	total = x + y	(ans: 7)
()	Parentheses	(x + 2) * y	(ans: 14)



ARITHMETIC OPERATORS

Changing a variable value

Example:

```
counter = 1
```

```
print( counter )
```

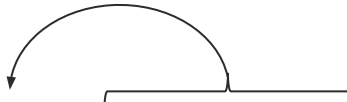
<- this will display 1

```
print( counter + 1 )
```

<- this will take the **value** of counter and add 1 to it and it will display the result 2 but counter itself is not changed.

```
print( counter )
```

<- counter is not changed so this will display 1



```
counter = counter + 1
```

<- saves the result back into counter. this increases counter by 1

```
print( counter )
```

<- counter is changed so it will display 2

ARITHMETIC OPERATORS

You can use the following **shortcuts** for arithmetic operations

Operator	Example		
	x = 5 and y = 2		
+=	x += y	is the same as x = x + y	(x is increased by y and x becomes 7)
-=	x -= y	is the same as x = x - y	(x is decreased by y and x becomes 3)
*=	x *= y	is the same as x = x * y	(x is multiply by y and x becomes 10)
/=	x /= y	is the same as x = x / y	(x is divided by y and x becomes a float 2.5)



ARITHMETIC OPERATORS

You can treat integer as part of float (but not the other way round).

In an operation, when there is a mix of integers and floats, the result will be a float.

Example:

```
x = 5      ← x is integer
y = 2      ← y is integer
z = x + y  ← z is integer
print(z)
```

Screen output

7

```
x = 5      ← x is integer
y = 2.0     ← y is float
z = x + y   ← z is float
print(z)
```

Screen output

7.0

ARITHMETIC OPERATORS

You can treat integer as part of float (but not the other way round).

Note that when you divide integers, the result will be a float.

Example:

```
x = 4      ← x is integer
y = 2      ← y is integer
z = x / y  ← z is float because of the division
print(z)
```

Screen output

2.0

```
x = 5      ← x is integer
y = 2      ← y is integer
z = x / y  ← z is float because of the division
print(z)
```

Screen output

2.5

STRING OPERATORS

You can also perform operations on strings.

Operator	Description	Example	
+	Concatenate 2 strings	"ABC" + "DEF"	(result: 'ABCDEF')
*	Repeat string by integer	"ABC" * 3	(result: 'ABCABCABC')
()	Prioritise with Parentheses	("A" + "B") * 2 vs "A" + ("B" * 2)	(result: 'ABAB' vs 'ABB')
=	Assign	newstring = "A" + "B" * 3	(result: 'ABBB')

Example:

```
salutation = "Mrs."  
name = "Tay"  
print( "Name:", salutation + name )    <- this will display Name: Mrs.Tay
```



TYPE CASTING

CASTING

In order to perform certain operations, you need to cast them to the right data type. For example, you can only divide numbers and not string.

`value = 9` ← The variable named *value* is assigned an integer

`value = str(value)` ← This changes it to a string '9'

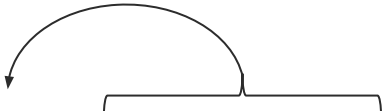
`value = int(value)` ← This changes it back to integer 9
To cast to integer, value must contain only digits

`value = float(value)` ← This changes it to a float 9.0
To cast to float, value must contain only digits or with one decimal point (e.g. 9, 9.0, 9.123, etc)

CASTING

Note that casting does not change the variable, it only takes the value and return the cast value. Below shows an example.

<code>num = 9</code>	← The variable num is assigned an integer 9
<code>print(num)</code>	← This displays 9
<code>print(float(num))</code>	← <code>float(num)</code> converts the value and returns 9.0 but does not change the variable num It displays the result 9.0
<code>print(num)</code>	← As num remains as integer, it displays 9

	
<code>num = float(num)</code>	← To change the variable type, you need to assign it back.
<code>print(num)</code>	← Now num is a float type with value 9.0

INPUT

INPUT

Input command allows the user to enter data from the keyboard. When user hit the enter key, it will return the data entered as a *STRING*. Use a variable to save the value.

text message to display on the screen
↓
input()

Example:

name = input("Enter your name: ")
print(name)

Use a variable to
save the input value

Screen output

Entered by user

Enter your name: *Bert*
Bert

INPUT CASTING

As input command always returns a string, you need to perform casting to the right datatype before performing operations (e.g. if you need to do arithmetic operation, then cast it to integer or float).

Example:

```
num = input("Enter number: ")  
num = int( num )  
num = num * 2  
print("Double the number is", num)
```

← Cast to integer first to do math operation

Screen output

Entered by user

Enter number: 5
Double the number is 10

INPUT DISPLAY

For input command, you cannot use comma to join different strings or variables to form the display message. You need to use the plus operator (+). You can only add up strings in this case. Hence, cast any non-string value to strings.

Example:

to join 3 parts of text with +, all of them has to be strings

```
num = 3  
name = input( "Enter name of player #" + str(num) + " : " )
```

num is an integer so cast to string in order to join up all the text

Screen output

Enter name of player #3:

INPUT DISPLAY

Alternatively, you can use f-string formatting as shown below

Example:

```
num = 3  
name = input( f"Enter name of player #{num}: " )
```



Place variable in curly brackets.
Casting is not required.

Screen output

Enter name of player #3:

TypeError:

You when receive the a type error message show below, it means that you are trying to perform an operation that is not valid for the current value type.

TypeError: unsupported operand type(s) for -: 'str' and 'int'

The most common occurrence of this error is to forget to cast variables to numerical types after the input. Below shows an example:

```
n = input("Enter a num:")  
n = n - 1
```

need to cast n to integer or float

ValueError:

You when receive the a value error message show below, it means you are trying to perform an invalid casting of type.

ValueError: invalid literal for int() with base 10

The most common occurrence of this error is to forget to cast variables to numerical types after the input. Below shows an example:

```
name = input("Enter a name:")  
name = int(name)
```

name is a not a number so it cannot be cast to integer

SPLITTING STRING

You can enter multiple values in a single row and separate them using ***split()*** command.

Example:

```
name = input( "Enter 3 words: " )  
text1, text2, text3 = name.split( )  
print(text1)  
print(text2)  
print(text3)
```

The command will search for spaces in the string and split it into different strings

you need to have exactly the same number of variables as the split strings

Screen output

```
Enter 3 words: red green blue  
red  
green  
blue
```

SPLITTING STRING

You can use a separator string with the ***split()*** command to split as well.

Example:

```
name = input( "Enter a string: " )  
→ text1, text2, text3 = name.split( "xx" )  
  print(text1)  
  print(text2)  
  print(text3)
```

↓
The command will search for
"xx" in the string and split it
into different strings

you need to have exactly the same
number of variables as the split strings

Screen output

```
Enter a string: RedxxGreenxxBlue  
Red  
Green  
Blue
```

IF STATEMENT

CONDITION

A condition is a statement that can only be **True** or **False** (call boolean) and nothing else. It is similar to a Yes and No. If it returns something else, it is not a condition.

Are the following conditions (i.e. the answer to them is either a True or False)?

mom is at home

she is taller than him

her age is the same as his brother's

the cat is hungry and there is no food

I have \$5

\$3 + \$2

x is greater than y

x is equal to 5

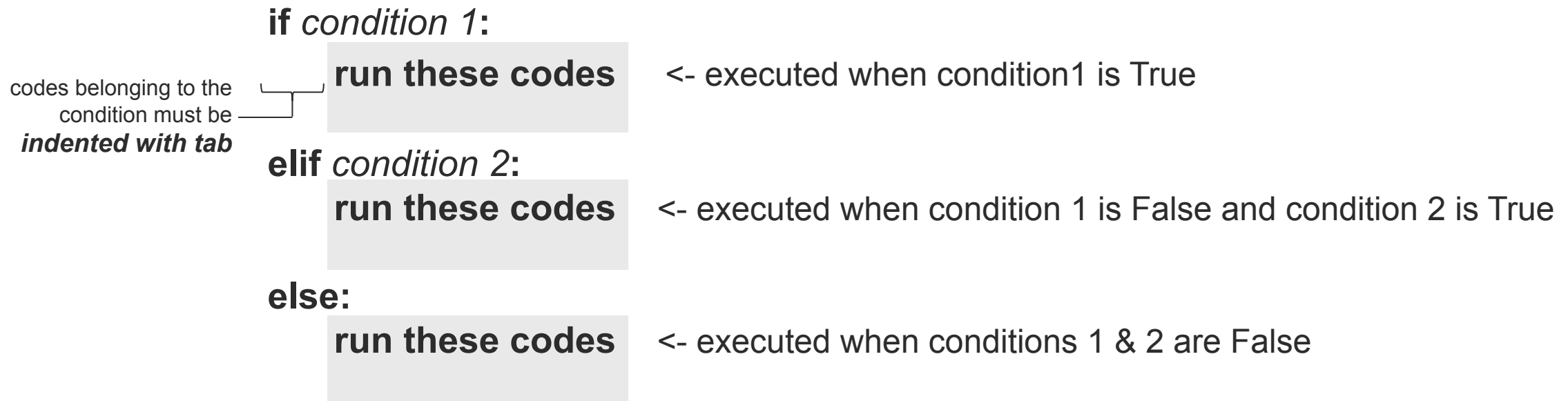
(x + y) is less than 10

x - 2



IF STATEMENT

An if-statement checks if a condition (or multiple conditions) is true, then it will execute some codes.



You can have many *elif* in an *if* block. **When a condition is True and the codes are executed, any remaining *elif* and *else* will be skipped.**

COMPARISON OPERATORS

	Comparison Operators
x is equal to y	<code>x == y</code>
x is not equal to y	<code>x != y</code>
x is smaller than y	<code>x < y</code>
x is smaller or equal to y	<code>x <= y</code>
x is greater than y	<code>x > y</code>
x is greater or equal to y	<code>x >= y</code>
x+1 is less than or equal to y/4	<code>(x + 1) <= (y / 4)</code>
x is divisible by y	<code>(x % y) == 0</code>



IF STATEMENT

What will be displayed in the following?

```
x = 3
if x < 5:
    print("x is smaller than 5")
else:
    print("x is not smaller than 5")
```

IF STATEMENT

What will be displayed in the following?

```
x = 3
y = 6
if x < 5:
    print("x is smaller than 5")
elif y > 5:
    print("y is greater than 5")
else:
    print("x is not smaller and y is not greater than 5")
```

IF STATEMENT

What will be displayed in the following?

```
x = 3
y = 6
if x < 5:
    print("x is smaller than 5")
if y > 5:
    print("y is greater than 5")
else:
    print("y is not greater than 5")
```

Multiple conditions: AND / OR / NOT

if condition 1 **and** condition 2:

run these codes

← do when both conditions are True

if condition 1 **or** condition 2:

run these codes

← do when either one or both conditions are True

if **not** condition 1:

run these codes

← do when condition 1 is False

if (condition 1 **or** condition 2) **and** condition 3:

run these codes

← do when conditions 1 or 2 is True and 3 is also True

BOOLEAN

You can make a variable a ***boolean*** data type so that it stores either ***True*** or ***False***.

Example:

night = ***False***

↓

this variable is a boolean when you assign it ***True*** or ***False*** (first letter capital).

if night: ← You can also use "***if night == True***"
 print("Time to rest")
else:
 print("Time to play")

Nested If

Example:

if condition 1:

if condition 2:

indented with tab

indented with tab

run codes A

else:

run codes B

else:

if condition 3:

run codes C

else:

run codes D

	Condition 1	Condition 2	Condition 3	Which codes will run (A/B/C/D) ?
1	False	False	False	
2	False	False	True	
3	False	True	False	
4	False	True	True	
5	True	False	False	
6	True	False	True	
7	True	True	False	
8	True	True	True	



Nested If

Example:

if condition 1:

 if condition 2:

indented with tab

indented with tab run codes A

 else:

 run codes B

else:

 if condition 3:

 run codes C

 else:

 run codes D

	Condition 1	Condition 2	Condition 3	Which codes will run (A/B/C/D) ?
1	False	False	False	D
2	False	False	True	C
3	False	True	False	D
4	False	True	True	C
5	True	False	False	B
6	True	False	True	B
7	True	True	False	A
8	True	True	True	A



RANGE

Checking range is common in coding.

Below are examples showing how you check for **continual** ranges using **and** operator:

Examples:

given num is an integer

if num > 0 **and** num < 5: *checking if num is in range 1,2,3,4*

...

if num >= 0 **and** num <= 5: *checking if num is in range 0,1,2,3,4,5*

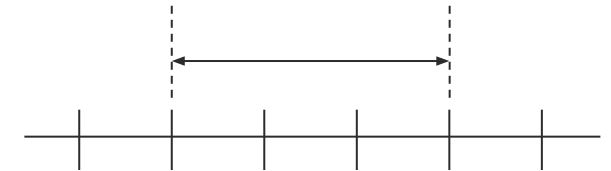
...

if num >= 0 **and** num < 5: *checking if num is in range 0,1,2,3,4*

...

if num > 0 **and** num <= 5: *checking if num is in range 1,2,3,4,5*

...



RANGE

Checking range is common in coding.

Below are examples showing how you check for **disjoint** ranges using **or** operator:

Examples:

given num is an integer

if num < 5 **or** num > 10:

...

*checking if num is in one of the ranges:
4 and below or 11 and above*

if num <= 5 **or** num >= 10:

...

*checking if num is in one of the ranges:
5 and below or 10 and above*

if num <= 5 **or** num > 10:

...

*checking if num is in one of the ranges:
5 and below or 11 and above*

if num < 5 **or** num >= 10:

...

*checking if num is in one of the ranges:
4 and below or 10 and above*



LOOP

for-loop with range()

FOR-LOOP

A *for-loop* is used to repeat some codes for certain range.

It uses a variable to store the count as it loops.

variable will start counting from this number

variable will stop counting at end-1 (it will not reach end)

for each count, add step to count

for *variable* in range(*start*, *end*, *step*):

run these codes

codes belonging to the loop must be ***indented with tab***

Examples

Run these codes to see what is displayed.

- | | |
|--|--|
| <pre>for i in range(0, 10, 1):
 print(i)</pre> | ← As it loops, i will count 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 |
| <pre>for i in range(5, 10, 2):
 print(i)</pre> | ← As it loops, i will count 5, 7, 9 (add 2 each count) |
| <pre>for i in range(10, 1, -1):
 print(i)</pre> | ← This is a countdown. You <i>must indicate negative step</i> for countdown.
As it loops, i will count 10, 9, 8, 7, 6, 5, 4, 3, 2 |
| <pre>for i in range(5, -7, -3):
 print(i)</pre> | ← As it loops i will count 5, 2, -1, -4 (add -3 each count) |
| <pre>for i in range(3, 10):
 print(i)</pre> | ← If you indicate only 2 values, they will be taken as the <i>start</i> and <i>end</i> with default <i>step</i> 1. As it loops, i will count 3, 4, 5, 6, 7, 8, 9 |
| <pre>for i in range(5):
 print(i)</pre> | ← If you indicate only 1 value, it will be taken as the <i>end</i> with default <i>start</i> 0 and <i>step</i> 1. As it loops, i will count 0, 1, 2, 3, 4 |



FOR-LOOP

You can use variables in the loop range.

Examples:

```
start = 10
```

```
end = 50
```

```
step = 3
```

```
for i in range(start, end, step):  
    print(i)
```

Nested For-Loop

Example:

```
for i in range(3):  
    print( "---- i is", i )  
    for k in range(1, 4):  
        print( "k is", k )
```

use a different variable
for each inner loop

Observe the *indentation* (spacing in front of the codes).

Screen output

```
---- i is 0  
k is 1  
k is 2  
k is 3  
---- i is 1  
k is 1  
k is 2  
k is 3  
---- i is 2  
k is 1  
k is 2  
k is 3
```

Breaking For-Loop

You can use the ***break*** command to exit a for-loop anytime.

Example:

```
for i in range(10):  
    if i == 5:  
        break  
    else:  
        print(i)
```

Screen output

```
0  
1  
2  
3  
4
```

Printing in a single row

Each time when you use ***print()***, the cursor will go to the next line. Below shows an example of how you can use a string variable to print values in a single row.

Method 1: using a variable

```
line = ""  
for i in range(5):  
    line = line + str(i)  # append to variable  
print(line)
```

Method 2: using print with end

```
for i in range(5):  
    print( i, end="" ) # keep the cursor on the same row after printing
```

Screen output

01234

GOOD PROGRAMMING HABITS

It is important to develop good programming habits to make your codes well organized and readable because you are likely to be working as a team and your teammates will read your codes. It will also be beneficial to yourself should you need to come back and read a code that you have written a long time back. Below are 2 versions of the same codes.

Which of the following is easier to understand (they are the same codes)?

Version 1

```
a= 25
b = 50
c  = 12
d =40
If (a*b )> ( c* d):
    print ("1")
else:
    print(" 2")
```

Version 2:

```
# set length and breadth of rectangle 1
length1 = 25
breadth1 = 50

# set length and breadth of rectangle 2
length2 = 12
breadth2 = 40

# check which rectangle has a bigger area
if (length1 * breadth1) > (length2 * breadth2):
    print("Rectangle 1 has a bigger area")
else:
    print("Rectangle 2 has a bigger area")
```

Use **MEANINGFUL NAMES** for variables

Version 2:

set length and breadth of rectangle 1

length1 = 25

breadth1 = 50

set length and breadth of rectangle 2

length2 = 12

breadth2 = 40

check which rectangle has a bigger area

if (length1 * breadth1) > (length2 * breadth2):

print("Rectangle 1 has a bigger area")

else:

print("Rectangle 2 has a bigger area")

Add short **COMMENTS** to them using '#'

Version 2:

set length and breadth of rectangle 1

length1 = 25

breadth1 = 50

set length and breadth of rectangle 2

length2 = 12

breadth2 = 40

check which rectangle has a bigger area

if (length1 * breadth1) > (length2 * breadth2):

print("Rectangle 1 has a bigger area")

else:

print("Rectangle 2 has a bigger area")

Add **LINE BREAKS** to break the codes into parts

Version 2:

set length and breadth of rectangle 1

length1 = 25

breadth1 = 50

set length and breadth of rectangle 2

length2 = 12

breadth2 = 40

check which rectangle has a bigger area

if (length1 * breadth1) > (length2 * breadth2):

print("Rectangle 1 has a bigger area")

else:

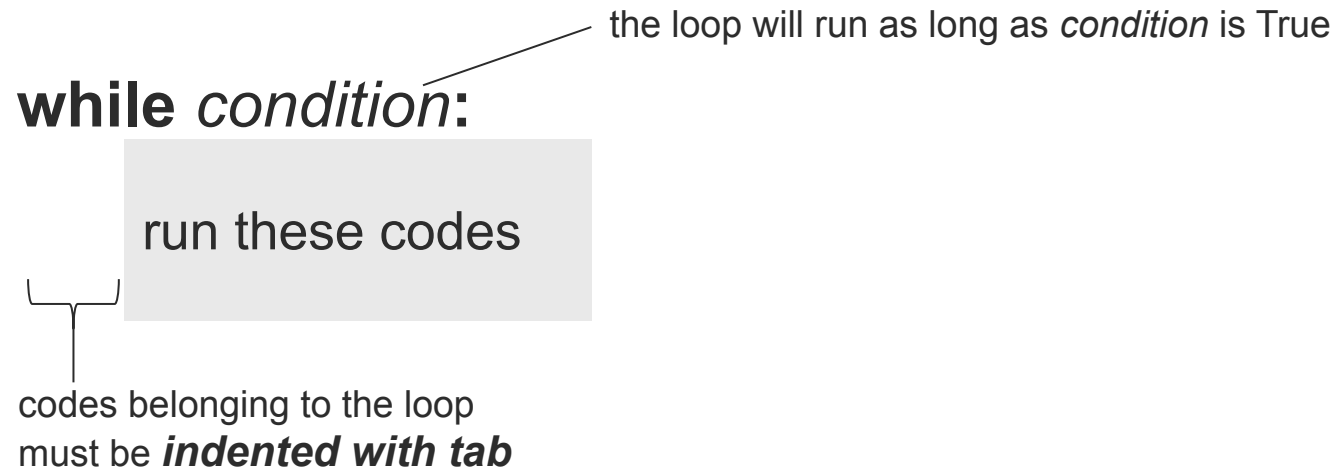
print("Rectangle 2 has a bigger area")

LOOP

while-loop

WHILE-LOOP

If you do not know how many times to repeat, *while-loop* will be suitable as it can either repeat forever or terminate upon a condition.



WHILE-LOOP

Example:

What number will be displayed when you run the codes below?

```
counter = 0
while counter < 10:
    print(counter)
    counter = counter + 1
```

WHILE-LOOP (infinite loop / break / continue)

Example:

```
counter = 0
```

```
while True:
```

 ← This will form an infinite loop as the condition is always True

```
    counter = counter + 1
```

```
    if counter < 10:
```

```
        continue ← 'continue' will cause program to jump to start of loop
```

```
    elif counter == 100:
```

```
        break ← 'break' will cause program to exit nearest loop
```

```
    print(counter)
```

```
print("I am out of the while loop")
```

Nested While-Loop

Example:

```
outer = 0
```

```
while outer < 3:
```

```
    print( "---- outer is", outer )
```

```
    inner = 0
```

```
    while inner < 2:
```

```
        print( "inner is", inner )
```

```
        inner = inner + 1
```

```
    outer = outer + 1
```

Observe the *indentation* (spacing in front of the codes).

Screen output

```
---- outer is 0  
inner is 0  
inner is 1  
---- outer is 1  
inner is 0  
inner is 1  
---- outer is 2  
inner is 0  
inner is 1
```

LIST

basics

LIST

A list is used to store a *group* of values.

Each value is accessed by an index that starts from zero.

A list has square brackets with commas separating each value.

	Index (position)
	0 1 2 3
<i>mylist</i>	= [value , value , value , value]

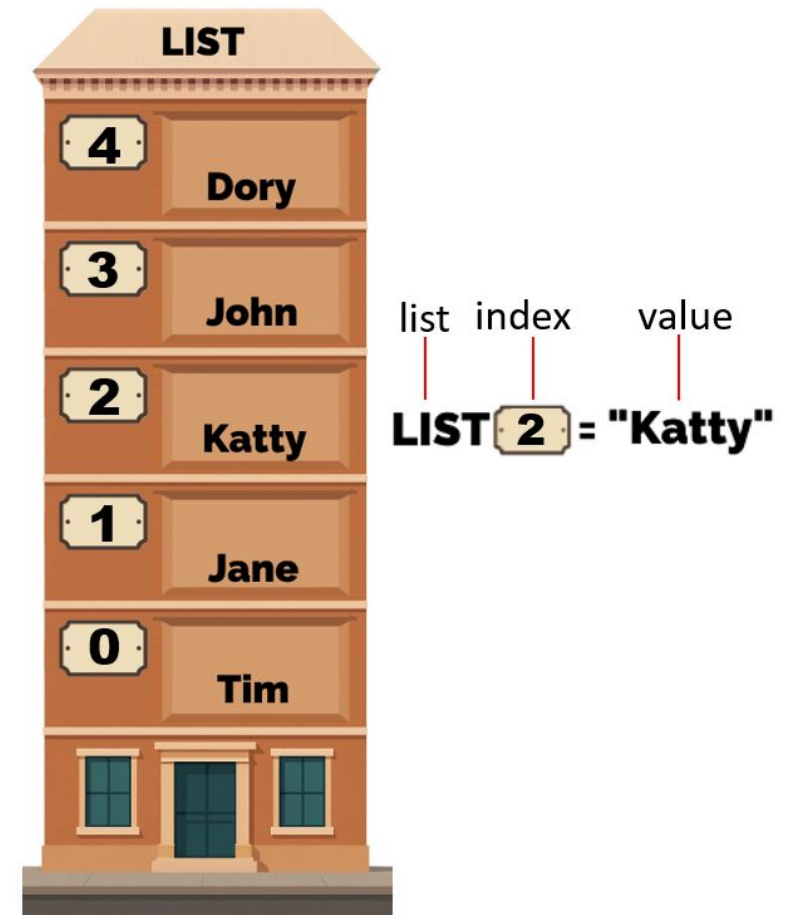
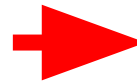
Index value is the position (starts from 0) and
mylist[index value] contains the value at that position.

List

To help in remembering, you can think of it as a building (**list**) where each level (**index**) has a room that houses someone (**value**).

List	
[index]	
0	value 0
1	value 1
2	value 2
3	value 3
4	value 4

List[2] = value 2



LIST

Example:

a list can contain any data types. below is a list *name* storing strings.
`name = [ "Colt" , "Barley" , "Dynamike" ]`

accessing individual item
`print( name )`
`print( name[0] )`
`print( name[1] )`
`print( name[2] )`

Screen output

```
['Colt', 'Barley', 'Dynamike']  
Colt  
Barley  
Dynamike
```

LIST

List length

You can use the ***len()*** command to get the number of items in a list. It returns an integer.

num_of_items = ***len(mylist)***

The **last index** in a list is ***len(mylist) - 1***.

For example, if a list has 4 items, its index will run from 0 to 3 (3 is 4-1).

Index (position)

0 1 2 ***len(mylist) - 1***

mylist = [value , value , value , ... , value]

LIST

A list can store different data types.

A list can contain any data types.

```
mixed = [ 10 , "Colt" , 0.56 , True ]
```

accessing individual values of a list

```
print( mixed )  
print( mixed[0] )  
print( mixed[1] )  
print( mixed[2] )  
print( mixed[3] )
```

```
print( len(mixed) )  
print( mixed[ len(mixed)-1 ] )
```

Screen output

```
[10, 'Colt', 0.56 ,True]  
10  
Colt  
0.56  
True
```

```
4  
True
```

LIST

Negative index

You can also access the list using negative index starting from the end.
The last index is -1.

	Index (position)			
	- len(mylist)	-3	-2	-1
<i>mylist</i> =	[value , ... , value , value , value]			

mylist[-1] contains the last value (same as *mylist[len(mylist)-1]*)

LIST

Example:

a list can contain any data types. below is a list *name* storing strings.
`name = [ "Colt" , "Barley" , "Dynamike" ]`

accessing individual item
`print( name )`
`print( name[-1] )`
`print( name[-2] )`
`print( name[-3] )`

Screen output

```
['Colt', 'Barley', 'Dynamike']  
Dynamike  
Barley  
Colt
```

LIST – Looping through List

List is often used with for-loop as the loop counter can be used as the index to access each item.

Example

a list of names

```
name = [ "Colt" , "Barley" , "Dynamike" , "Shelly"]
```

Method 1: looping using index (i).

You can choose the range with this method.

```
for i in range( len(name) ): _____ len() in this case returns value 4.  
                                   So i will count from 0 to 3  
    print( i, name[ i ] ) _____ i is the index  
                                   name[i] is the value
```

Method 2: looping using value

```
for i in name:
```

```
    print( i ) _____ i is the value
```

Screen output

```
0 Colt  
1 Barley  
2 Dynamike  
3 Shelly
```

```
Colt  
Barley  
Dynamike  
Shelly
```

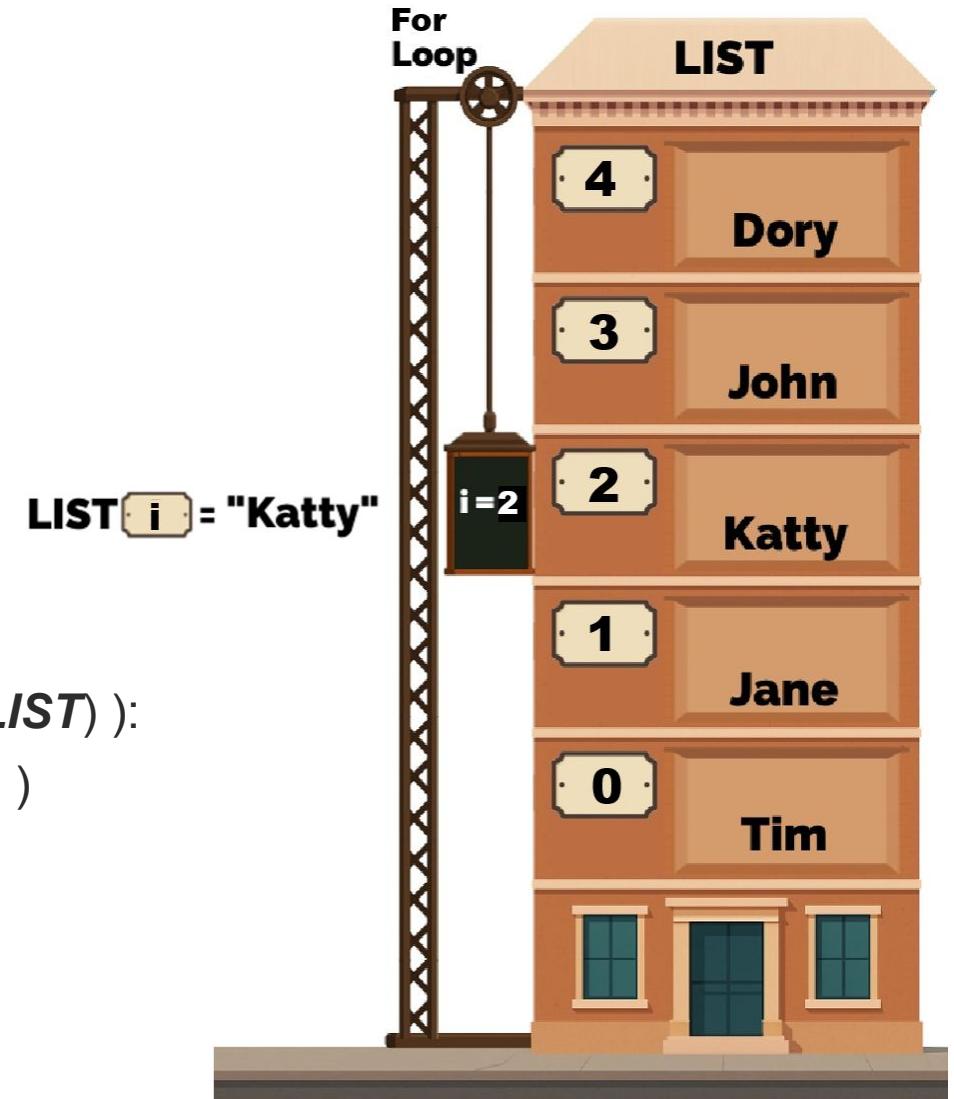
PYTHON CODING

LIST BASICS

LIST – Looping through List

Previously, we visualised a list as a building. Now, imagine a lift beside it, stopping at each floor, just like a For-loop counting from 0 to the top. As the lift ascends, it stores the current floor number in the counter variable, which acts as the index to access values in the list.

```
for i in range( len(LIST) ):
    print( LIST [ i ] )
```



LIST - Creating a list

How to create a list `['A', 'A', 'A']` ?

Method 1: declare and assign

```
mylist = ['A', 'A', 'A']
```

Method 2: using for-loop

```
mylist = ['A' for i in range(3)]
```

← number of items to create

Method 3: using `append()`

```
mylist = []
```

← you must create an empty list before appending

```
for i in range (3):
```

← number of items to create

```
    mylist.append('A')
```

IndexError:

You when receive the an index error message show below, it means you are trying to access a value from a list at an invalid index.

IndexError: list index out of range

Below shows an example:

```
names = ['Nigel', 'Angie', 'Sam' , 'Chloe']  
print(names[4])
```

index invalid (0 to 3)

LIST *manipulation*

LIST - Appending an item to a list

You can add an item to the end of a list by using the ***append()*** command.

```
list_variable.append( item )
```

Example:

```
mylist = [ "Red" , "Green" , "Blue" ]  
mylist.append( "Orange" )  
print( mylist )
```

Screen output

```
['Red', 'Green', 'Blue', 'Orange']
```

LIST - Deleting an item from a list

Use the ***pop()*** command to remove an item at an index from a list.

The removed item will be returned. You can use a variable to save it (optional).

Any items behind will be shifted up.

Note that popping an **invalid index** or when the list is **empty** will crash the program.

`list_variable.pop( index )`

Example:

If you don't need to save the popped value, you can simply use ***mylist.pop()***

```
mylist = [ "Red" , "Green" , "Blue" ]
removed = mylist.pop(1)
print( mylist )
print( removed )
mylist.pop() ← if you omit the index,
               it will remove the last item
print( mylist )
```

Screen output

```
['Red', 'Blue']
Green
['Red']
```

LIST - Inserting an item into a list

You can insert an item at an index of a list using ***insert()*** command.
The items at the index onwards will be shifted down.

list_variable.insert(index , item)

Example:

```
mylist = [ "Red" , "Green" , "Blue" ]  
mylist.insert( 1 , "Orange" )  
print(mylist)
```

Screen output

```
['Red', 'Orange', 'Green', 'Blue']
```

LIST - Assigning / editing items in a list

Treat each item in a list like a variable. You can perform variable operations on them based on their data type.

Example:

```
# list variable
```

```
num = [ 10 , 90 , 45 ]
```

```
# example of editing a value in a list
```

```
num[1] = num[0] + 7
```

```
print( num )
```

Screen output

```
[10, 17, 45]
```

LIST - Display

You can display a list without the punctuations.

Method 1:

```
mylist = [ "Red" , "Green" , "Blue" ]
```

```
print( mylist )
```

```
print( *mylist )
```

← *using * displays the content of a list
without the punctuations*

Method 2:

```
mylist = [ "Red" , "Green" , "Blue" ]
```

```
print( mylist )
```

append each value into a single string

```
line = ""
```

```
for value in mylist:
```

```
    line = line + value + " "
```

```
print( line )
```

Screen output

```
['Red', 'Green', 'Blue']  
Red Green Blue
```

LIST

You can check if a value exists in a list using the ***in*** or ***not in*** operator

Example:

```
mylist = [ "Green" , 123 ]
```

```
if "Green" in mylist:  
    print("found")
```

```
if 123 in mylist:  
    print("found")
```

```
if "something" not in mylist:  
    print("not found")
```

Screen output

found

found

not found

LIST - Creating a list by *splitting* a string

Example 1:

```
string = "cat dog mouse"  
wordlist = string.split()  
print( wordlist )
```

split() will search for spaces in the string and separate the content

Screen output

```
['cat', 'dog', 'mouse']
```

Example 2:

```
string = "cat"  
wordlist = string.split()  
print( wordlist )
```

if there is no space character in the string, it will just be a list of one value

Screen output

```
['cat']
```

LIST - Finding item in a List

You can find the index of the **first occurrence** of an item in a List using ***index()*** .

```
mylist = [ "Aa" , "Bb" , "Cc" , "Dd" , "Cc" ]  
print( mylist.index("Cc") ) ← this will display 2 (index of the first occurrence)
```

However, `index()` command will crash the program if the item does not exist.
Hence, you need to check for existence first using ***in*** before using it.

```
mylist = [ "123" , 0 , "5Z" , 42 , "Ee" ]  
if "42" in mylist:  
    print( mylist.index("42") )
```

Will the above print anything when run?

STRING MANIPULATION

Reading a Character from String

String can be read in the same manner as **List** using index

```
0 1 2 3 4 5 6  <- index starts from 0
text = "ABCDEFGH"
print( text[0] )    this will display "A"
print( text[4] )    this will display "E"
```

Negative index is used to read from the end of the string

```
-7 -6 -5 -4 -3 -2 -1  <- index starts at -1 from the end
text = "ABCDEFGH"
print( text[-1] )    this will display "H"
print( text[-3] )    this will display "F"
```

len() can be used to find out the length of the string, that is number of characters (including invisible ones)

```
text = "ABCDEFGH"
print( len(text) )    this will display 8
```

Reading a Character from String

Similar to List, you can use a for-loop to access each character

Method 1

```
text = "Hello, world!"  
for i in range( len(text) ):  
    print( text[i] )
```

Method 2

```
text = "Hello, world!"  
for i in text:  
    print( i )
```

Screen output

```
H  
e  
l  
l  
o  
,  
  
w  
o  
r  
l  
d  
!
```

Extracting Substring from String

Substring can be extracted using range in index [*start* : *end+1*]

```
      0 1 2 3 4 5 6
text = "ABCDEFGH"
      -7 -6 -5 -4 -3 -2 -1
```

`print( text[0:2] )` *this will display "AB" (similar to range, the end index is offset by 1)*

`print( text[3:6] )` *this will display "DEF"*

`print( text[-6:-3] )` *this will display "BCD"*

`print( text[3:] )` *this will display "DEFG" (from start index till end)*

`print( text[:3] )` *this will display "ABC" (from beginning till end index less 1)*

Note that you **cannot** change individual character of a string using assignment.

```
text = "ABCDE"
text[0] = "Z"    this command is INVALID as you cannot change the character in a string directly
```

You may change it to a list to edit the content.

Casting String to List of characters

```
text = "ABCDE"
charlist = list( text )    charlist will be ["A", "B", "C", "D", "E"]
```

Converting List to String

```
stringlist = [ "A", "BCD" , "E" ]
text = "".join( stringlist )    text will be "ABCDE"
```

Checking if a Substring exists in a String

You can use the ***in*** operator to check if a substring exists in string

```
text = "ABCDEFGFG"
```

```
if "A" in text:
```

```
    print( "found" )    this will this display "found"
```

```
if "CDE" in text:
```

```
    print( "found" )    this will this display "found"
```

Find the position of a Substring in a String

You can use the ***find()*** command to find the index of the first occurrence of a substring in a string. It will return -1 if not found.

Example:

```
sentence = "Hello, world."  
substring = "world"
```

```
idx = sentence.find(substring)
```

```
if idx != -1:  
    print("Substring found")  
else:  
    print("Substring not found.")
```

MISCELLANEOUS

Page	
96	<u>common error messages</u>
97	<u>clear screen</u>
98	<u>in operator</u>
99	<u>print() with end / sep</u>
100	<u>zfill()</u>
101	<u>casting and join()</u>
102	<u>round()</u>
105	<u>split() / strip() / lstrip() /rstrip()</u>
106	<u>lower() / upper() / title() / isdigit() / isalpha()</u>
107	<u>ord() / chr()</u>
108	<u>index() / find()</u>
109	<u>sleep() / random() / math</u>



Common Error Messages

SyntaxError: invalid syntax

Invalid code structure. Do you missing brackets, quotes or uses "=" instead of "==" for comparison, etc?

TypeError: unsupported operand type(s) for -: 'str' and 'int'

Performing an operation that is not valid for the current value type. Did you forget to cast?

ValueError: invalid literal for int() with base 10

Performing invalid casting. If you are casting to integer or float, make sure the content is a valid number format.

IndexError: list index out of range

Accessing invalid index of a list. Check that the index of a list is in range (0 to len(list)-1)

TypeError: func() missing n required positional argument

TypeError: func() takes n positional arguments but m was given

The number of values that you pass to a function when calling it are not matching.

Clear Screen

For macOS and Linux

```
import os  
os.system('clear')
```

For Windows

```
import os  
os.system('cls')
```

Clear screen command is dependent on the IDE's implementation. Some IDEs do not support os module or the os clears screen command has no effect. For the purpose of creating animation, an old-school trick is by printing a lot of blank lines to push the display to the bottom, creating an illusion of screen being cleared.

```
print(" \n" * 40)
```

↖
space here

IN Operator

The 'in' operator can function in different ways:

for i **in** range(10): ← *for use with range*
 ...

for name **in** namelist: ← *for use with a list (or other iterables like dictionary which you will learn later). In this example, variable name will store each value in namelist as it loops instead of counting.*
 ...

if name **in** namelist: ← *for use in condition. In this example, "name in namelist" will return true if name exists in namelist. This works for other iterables.*
 ...

if substring **in** string: ← *for use in condition. In this example, "substring in string" will return true if substring exists in string.*
 ...

PRINT Features

```
text = ["A", "B", "C"]
```

```
print(text[0], end="") ← remove the newline after print  
print(text[1])         "B" is then joined to "A".  
print(text[2])
```

Screen output

```
AB  
C
```

```
for i in text:  
    print(i , end="") ← remove the newline after print
```

```
ABC
```

```
for i in text:  
    print(i , end=" ") ← you can also customize the end  
                        instead of the default newline
```

```
A B C
```

```
print(*text , sep="") ← remove the space between the values
```

```
ABC
```

```
print(*text , sep="--") ← you can also customize the separator  
                        instead of the default space
```

```
A--B--C
```

Leading and Trailing Zeroes:

```
num = 1  
print( str(num).zfill(4) )
```

Screen output

0001

If there are less than 4 characters, leading zeroes will be displayed

```
num = 1.2  
print( str(num).zfill(4) )
```

01.2

If there are less than 4 characters including decimal point, leading zeroes will be displayed

```
num = 1.2  
print( "%06.02f " %num )
```

001.20

If number after decimal point is less than 2 digits, trailing zeros will be displayed.

If there are less than 6 characters in total including decimal point, leading zeros will be displayed.

Type Casting

to integer	variable = int(variable)	<i>variable can be string or number But must contain digits only</i>
to float	variable = float(variable)	<i>variable can be string or number but must contain digits. May also contain decimal point.</i>
to string	variable = str(variable)	<i>variable can contain any strings/numbers</i>
to list of characters	variable = list(variable)	<i>variable must be a string</i>
to string	variable = "".join(variable)	<i>variable must a list of strings</i>
to string	variable = "".join([str(i) for i in variable])	<i>variable can be a list of any strings/numbers</i>



Rounding:

```
num = 1.234000
```

```
print( round(num) )
```

```
print( round(num, 0) )
```

```
print( round(num, 2) )
```

```
print( round(num, 6) )
```

*Indicate the number of
decimal points required*

Screen output

1

1.0

1.23

1.234

All trailing zeroes are dropped in round function.

About Python Round Method

Note about Python rounding:

In Python, the rounding method is a little different in the way they handle numbers that are exactly halfway between two possible rounded values.

In school, you could be taught the "round half up" method, where numbers that are exactly halfway will be rounded up. For examples, 1.5 rounds to 2 and 2.5 rounds to 3.

Python uses Banker's rounding ("round half to even"), where such numbers will be rounded to the nearest even number instead. For example, both 1.5 and 2.5 will be rounded to 2.

Banker's rounding are normally used in financial and statistical sectors to improve accuracy. In this curriculum, you do not have to worry about the precision issue due to the different rounding methods. The following slides will show you how to perform "round half up" for your interest.

How to Perform "round half up"

There are different ways to perform "round half up". One method is as shown, to round **value** to **dec** number of decimal places:

```
import math
multiplier = (10 ** dec)
rounded_number = math.floor(abs(value) * multiplier + 0.5) / multiplier
rounded_number = math.copysign(rounded_number, value)
```

*This removes negative sign so
that the rounding is consistent*

This restores the original sign

For example, in order to round 1.555 to 2 decimal points,

```
import math
multiplier = (10 ** 2)
rounded_number = math.floor(abs(1.555) * multiplier + 0.5) / multiplier
rounded_number = math.copysign(rounded_number, 1.555)
```

the result will be 1.56 (if you use `round(1.555, 2)`, it will produce 1.55).

Split / Strip:

`mystring = "Apple Orange"`
`str1, str2 = mystring.split()` ← *separate string where there is white space*
str1 == "Apple" and str2 == "Orange"

`mystring = "Apple---Orange"`
`str1, str2 = mystring.split("---")` ← *separate string where there is exists the specified string*
(ie. "---" in this example)
str1 == "Apple" and str2 == "Orange"

`mystring = "Apple Orange"`
`str_list = mystring.split()` ← *separate string where there is white space into a list*
str_list[0] == "Apple" and str_list[1] == "Orange"

`mystring = " Apple Orange "`
`mystring = mystring.strip()` ← *remove white spaces in front and at end of the string*
mystring == "Apple Orange"

`mystring = " Apple Orange "`
`mystring = mystring.lstrip()` ← *remove white spaces in front of the string*
mystring == "Apple Orange "

`mystring = " Apple Orange "`
`mystring = mystring.rstrip()` ← *remove white spaces at the end of the string*
mystring == " Apple Orange"



Alphanumeric:

change all to uppercase

```
mystring = "Abc dEf"
```

```
print( mystring.upper() )
```

Screen output

```
ABC DEF
```

change all to lowercase

```
print( mystring.lower() )
```

```
abc def
```

change all to first letter capital

```
print( mystring.title() )
```

```
Abc Def
```

check if all characters
are digits

```
print( mystring.isdigit() )
```

```
False
```

check if all characters
are alphabet (space is
not alphabet)

```
print( mystring.isalpha() )
```

```
False
```

Character code:

Returns the code number of a character

```
print( ord("A") )
```

Screen output

65

Returns the character of a code number

```
print( chr(65) )
```

A

String / List index:

Find index of a substring in a string

```
mystring = "abcdefghi"  
print( mystring.find("def") )
```

Screen output

3

Find index of a value in a list

```
mylist = ["abc", "def", "ghi"]  
print( mylist.index("def") )
```

1

Other Useful Libraries:

import time

time.sleep(n) ← *Delay for n number of seconds where n is a positive float number*

import random

random.seed(0) ← *This line is optional but can be added to generate the same random numbers each time you restart your code. Useful for testing.*

number = random.randint(n, m) ← *Generate random number from n to m where n and m are integers*

import math

circumference = 10 * math.pi ← *PI constant*

squareroot = math.sqrt(n) ← *Square root of a number*

PRACTICES and IDEs

PYTHON CODING

PRACTICES AND IDES

IDE

You can download and install Python in your laptop (it comes with an IDE called IDLE). You can also install IDEs such as PyCharm and Visual Studio Code for coding.

There are also many sites that provide online IDEs for you to write and test codes. However, they are managed by external hosts and are constantly updating. So, some things might work differently after sometime. Here are some online options:

<https://trinket.strivemath.org/>

- Supports all the functions required in the curriculum
- Screen is wider with login account
- May need reload sometimes if freezes

<https://python-playground.com/>

- In order to keep the screen refreshed while interacting with `input()` commands in Turtle, use `screen.update()` after drawing.

<https://www.programiz.com/python-programming/online-compiler/>

- For simple code
- Does not support file IO, Turtle and OS modules

<https://coder.csuk.io/coder/>

- Pop-up window for `input()`
- Does not support OS modules

<https://web.thelab.sg> (portal IDE)

- Pop-up window for `input()`, which can affect display.
- Avoid running infinite loop
- Does not support file IO and OS module
- Limited control over Turtle window
- Less clear in error messages



Home Practice

You can find many free online resources to sharpen your Python skills at home. Just search for keywords like “Python exercises.” Here are a few sample links to get started.

<https://www.geeksforgeeks.org/python/python-exercises-practice-questions-and-solutions/>

<https://www.practicepython.org/exercises/>

<https://www.codingame.com/multiplayer/clashofcode>